

ปฏิบัติการครั้งที่ 3

เรื่อง Socket & Threading

เวลาทำปฏิบัติการ

120 นาที

ผลลัพธ์การเรียนรู้ (Learning Outcomes)

เมื่อปฏิบัติการเสร็จสิ้น นักศึกษาจะสามารถ:

1. สร้าง TCP Server และ Client ที่สื่อสารกันผ่าน socket ได้
2. เข้าใจ blocking behavior ของ accept() และ recv() และผลกระทบต่อ server
3. ใช้ ThreadPoolExecutor สร้าง server ที่รับ client หลายคนพร้อมกันได้
4. ใช้ Lock ป้องกัน Race Condition เมื่อมี shared state ระหว่าง threads
5. ออกแบบและพัฒนา Threaded Chat Room Server แบบ real-time ได้

เครื่องมือที่ใช้ในปฏิบัติการ

1. Python version 3.x up
2. Visual studio code
3. Terminal

ไฟล์ที่ต้องส่ง

task1_echo.py, task2_pool.py, task3_chat.py

แบบฝึกหัด

TCP Echo Server & Client

1

เวลา: 30 นาที | Skill: socket, bind, listen, accept, recv, sendall

โจทย์: สร้าง TCP Echo Server ที่รับข้อความจาก client แล้วส่งกลับเหมือนเดิม จากนั้นสร้าง client ที่ส่งข้อความ 3 ครั้ง และแสดงผล response

Server — task1_echo.py

เขียน TCP Server ที่มีพฤติกรรมดังนี้:

- bind บน 0.0.0.0 port 9000
- รับ connection จาก client 1 คน
- รับข้อความจาก client แล้วส่งกลับ (echo) จนกว่า client จะ disconnect
 - print [SERVER] Connected: <addr> เมื่อ client เชื่อมต่อ
 - print [SERVER] Disconnected เมื่อ client ตัดการเชื่อมต่อ

Source code ที่เตรียมไว้ให้:

```
# task1_echo.py — TCP Echo Server
import socket

HOST = '0.0.0.0'
PORT = 9000

# TODO: สร้าง socket, bind, listen
# TODO: รับ connection ด้วย accept()
# TODO: loop recv() และ sendall() จนกว่า client disconnect
```

Client — task1_echo.py (ต่อ)

เขียน TCP Client ที่ส่งข้อความ 3 ครั้ง แล้วแสดงผล response:

Source code ที่เตรียมไว้ให้:

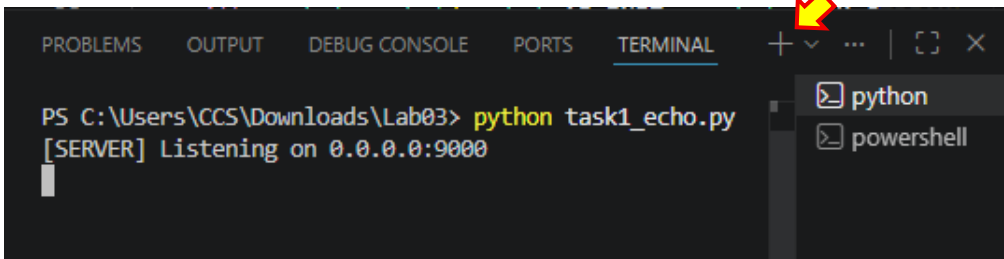
```
# Client — รับ terminal คนละหน้าต่าง
import socket

HOST = '127.0.0.1'
PORT = 9000
MESSAGES = ['Hello Server!', 'How are you?', 'Goodbye!']

# TODO: สร้าง socket และ connect
# TODO: loop ส่ง message แต่ละอัน และ print response
```

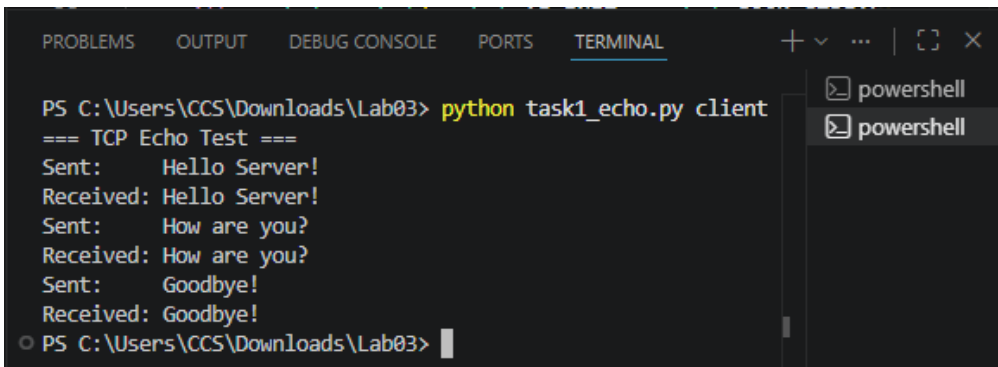
ผลลัพธ์ที่คาดหวัง

ผลลัพธ์ Terminal ฝั่ง Server ก่อนรันไฟล์ของ Client ***สามารถกด  เพื่อเพิ่มหน้าต่าง Terminal ได้



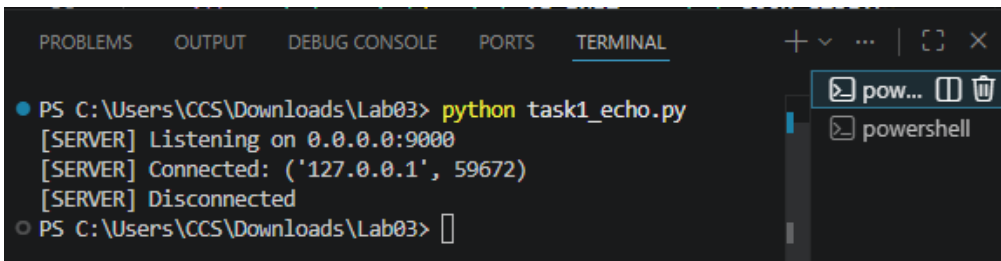
```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL + v ... | [ ] x
PS C:\Users\CCS\Downloads\Lab03> python task1_echo.py
[SERVER] Listening on 0.0.0.0:9000
```

ผลลัพธ์ Terminal เมื่อทำการรันของฝั่ง Client



```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL + v ... | [ ] x
PS C:\Users\CCS\Downloads\Lab03> python task1_echo.py client
=== TCP Echo Test ===
Sent: Hello Server!
Received: Hello Server!
Sent: How are you?
Received: How are you?
Sent: Goodbye!
Received: Goodbye!
PS C:\Users\CCS\Downloads\Lab03>
```

ผลลัพธ์ Terminal ฝั่ง Server หลังจากรันไฟล์ของ Client



```
PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL + v ... | [ ] x
● PS C:\Users\CCS\Downloads\Lab03> python task1_echo.py
[SERVER] Listening on 0.0.0.0:9000
[SERVER] Connected: ('127.0.0.1', 59672)
[SERVER] Disconnected
PS C:\Users\CCS\Downloads\Lab03>
```

Hint: รัน server ก่อน แล้วค่อยรัน client ใน terminal อีกหน้าต่าง — ถ้า server ไม่รับ client ให้ตรวจสอบว่า accept() ถูกเรียกก่อน connect()

เงื่อนไขการตรวจผ่าน

- ✓ ใช้ socket.socket(AF_INET, SOCK_STREAM)
- ✓ ใช้ SO_REUSEADDR เพื่อป้องกัน 'Address already in use'
- ✓ Server แสดง [SERVER] Connected: และ [SERVER] Disconnected
- ✓ Client แสดง Sent: และ Received: ทุกครั้ง
- ✓ ปิด socket ด้วย with block หรือ finally

แบบฝึกหัด

Multi-client Server ด้วย ThreadPoolExecutor

2

เวลา: 40 นาที | Skill: threading, ThreadPoolExecutor, SO_REUSEADDR

โจทย์: Task 1 รับได้แค่ client คนเดียว เพราะ `accept()` และ `recv()` เป็น blocking call — ถ้า client 2 เชื่อมต่อมา ขณะที่ server กำลังคุยกับ client 1 อยู่ จะต้องรอคิว

ในงานนี้ให้แก้ปัญหาโดยใช้ `ThreadPoolExecutor` รับ client ได้หลายคนพร้อมกัน

task2_pool.py

Source code ที่เตรียมไว้ให้:

```
# task2_pool.py - Multi-client Echo Server
import socket
from concurrent.futures import ThreadPoolExecutor

HOST = '0.0.0.0'
PORT = 9001
MAX_WORKERS = 10

def handle_client(conn, addr):
    # TODO: print connected, loop recv/sendall, print disconnected
    pass

# TODO: สร้าง server socket
# TODO: ใช้ ThreadPoolExecutor(max_workers=MAX_WORKERS)
# TODO: loop accept() และ pool.submit(handle_client, conn, addr)
```

ทดสอบ Multi-client

เปิด terminal 4 หน้าต่าง รัน server 1 หน้าต่าง และ client 3 หน้าต่างพร้อมกัน:

Source code ที่เตรียมไว้ให้:

```
# Client สำหรับทดสอบ (รัน 3 ตัวพร้อมกัน)
import socket, time

HOST, PORT = '127.0.0.1', 9001

with socket.socket() as s:
    s.connect((HOST, PORT))
    for i in range(3):
        msg = f'Message {i+1} from client'
        s.sendall(msg.encode())
        print('Echo:', s.recv(1024).decode())
        time.sleep(1)
```

ผลลัพธ์ที่คาดหวัง

ผลลัพธ์ Terminal ฝั่ง Server ก่อนรันไฟล์ Client :

```
PS C:\Users\point\OneDrive - Suranaree University of Technology\เอกสารการเรียนการสอน\1101180 - DGT01 1000 Network module\DGT01 1020 - network programming\Lab03\fileLab\Lab03>
python .\task2_pool.py
[SERVER] Listening on 0.0.0.0:9001 (max 10 workers)
```

ผลลัพธ์ Terminal เมื่อรันฝั่ง Client 1 แล้ว:

```
PS C:\Users\point\OneDrive - Suranaree University of Technology\เอกสารการเรียนการสอน\1101180 - DGT01 1000 Network module\DGT01 1020 - network programming\Lab03\fileLab\Lab03> echo "Client01"
Client01
PS C:\Users\point\OneDrive - Suranaree University of Technology\เอกสารการเรียนการสอน\1101180 - DGT01 1000 Network module\DGT01 1020 - network programming\Lab03\fileLab\Lab03> python .\task2_pool_client.py
Echo: Message 1 from client
Echo: Message 2 from client
Echo: Message 3 from client
PS C:\Users\point\OneDrive - Suranaree University of Technology\เอกสารการเรียนการสอน\1101180 - DGT01 1000 Network module\DGT01 1020 - network programming\Lab03\fileLab\Lab03> 
```

ผลลัพธ์ Terminal เมื่อรันฝั่ง Client 2 แล้ว:

```
PS C:\Users\point\OneDrive - Suranaree University of Technology> .\task2_pool_client.py
00 Network module\DGTO1 1020 - network programming\
Lab03\fileLab\Lab03> echo "Client02"
Client02
PS C:\Users\point\OneDrive - Suranaree University of Technology> .\task2_pool_client.py
00 Network module\DGTO1 1020 - network programming\
Lab03\fileLab\Lab03> python .\task2_pool_client.py
Echo: Message 1 from client
Echo: Message 2 from client
Echo: Message 3 from client
PS C:\Users\point\OneDrive - Suranaree University of Technology> .\task2_pool_client.py
00 Network module\DGTO1 1020 - network programming\
Lab03\fileLab\Lab03> 
```

ผลลัพธ์ Terminal เมื่อรันฝั่ง Client 3 แล้ว:

```
PS C:\Users\point\OneDrive - Suranaree University of Technology\เอกสารการเรียนการสอน\1101180 - DGT01 1000 Network module\DGT01 1020 - network programming\Lab03\fileLab\Lab03> echo "Client03"
Client03
PS C:\Users\point\OneDrive - Suranaree University of Technology\เอกสารการเรียนการสอน\1101180 - DGT01 1000 Network module\DGT01 1020 - network programming\Lab03\fileLab\Lab03> python .\task2_pool_client.py
Echo: Message 1 from client
Echo: Message 2 from client
Echo: Message 3 from client
PS C:\Users\point\OneDrive - Suranaree University of Technology\เอกสารการเรียนการสอน\1101180 - DGT01 1000 Network module\DGT01 1020 - network programming\Lab03\fileLab\Lab03>
```

ผลลัพธ์ Terminal ผัง Server ที่รันไฟล์ Client แล้ว:

```

PS C:\Users\point\OneDrive - Suranaree University of Technology\เอกสารการเรียนการสอน\1101180 - DGT01 1000 Network module\DGT01 1020 - network programming\Lab03\fileLab\Lab03> echo "Server"

Server

PS C:\Users\point\OneDrive - Suranaree University of Technology\เอกสารการเรียนการสอน\1101180 - DGT01 1000 Network module\DGT01 1020 - network programming\Lab03\fileLab\Lab03> python .\task2_pool.py
[SERVER] Listening on 0.0.0.0:9001 (max 10 workers)
[SERVER] Connected: ('127.0.0.1', 55946)
[SERVER] Connected: ('127.0.0.1', 55947)
[SERVER] Connected: ('127.0.0.1', 55948)
[SERVER] Disconnected: ('127.0.0.1', 55946)
[SERVER] Disconnected: ('127.0.0.1', 55947)
[SERVER] Disconnected: ('127.0.0.1', 55948)

```

***ให้กตคำสั่งรันฝั่ง Client พร้อมกัน เพื่อความแม่นยำของผลลัพธ์

เงื่อนไขผ่าน

- ✓ ใช้ ThreadPoolExecutor(max_workers=10)
- ✓ ใช้ pool.submit() ส่ง handle_client เข้า pool
- ✓ handle_client รัน loop recv/sendall ในแต่ละ thread
- ✓ Server รับ 3 clients พร้อมกันได้โดยไม่ต้องรอคิว
- ✓ แสดง [SERVER] Connected: <addr> และ Disconnected: <addr>

Note: สังเกตว่า handle_client() เหมือนกับ Task 1 ทุกอย่าง — สิ่งที่เปลี่ยนคือ main loop เท่านั้น นี่คือข้อดีของ ThreadPoolExecutor

แบบฝึกหัด

3

Threaded Chat Room Server

เวลา: 50 นาที | Skill: Lock, shared state, broadcast, client tracking

โจทย์: ต่อยอดจาก Task 2 สร้าง Chat Room Server ที่รองรับหลายคนพร้อมกัน โดยมีการ broadcast ข้อความจาก client คนหนึ่งไปยังทุกคนในห้อง

Requirements

R	Feature
R1	client เชื่อมต่อแล้วส่ง nickname ก่อน เช่น Alice
R2	ข้อความจาก client ถูก broadcast ไปยังทุกคนในรูปแบบ Alice: Hello!
R3	เมื่อ client เข้า/ออก ทุกคนได้รับแจ้ง เช่น [Alice joined] [Alice left]
R4	ใช้ Lock ป้องกัน Race Condition เมื่อแก้ clients dictionary
R5	รองรับอย่างน้อย 5 clients พร้อมกัน

task3_chat.py — Skeleton

```
# task3_chat.py - Threaded Chat Room Server
import socket, threading
from concurrent.futures import ThreadPoolExecutor

HOST = '0.0.0.0'
PORT = 9002

clients = {} # conn -> nickname
lock = threading.Lock()

def broadcast(msg, sender=None):
    # TODO: ส่ง msg ไปยัง clients ทุกคน ยกเว้น sender
    # TODO: ใช้ lock เมื่ออ่าน clients dict
    pass

def handle(conn, addr):
    # TODO: รับ nickname จาก client
    # TODO: เพิ่ม conn เข้า clients dict (ใช้ lock)
    # TODO: broadcast ทั่ว [nickname joined]
    # TODO: loop รับข้อความ แล้ว broadcast
    # TODO: เมื่อ disconnect ลบออกจาก dict และ broadcast [nickname left]
    pass

# TODO: สร้าง server socket และ ThreadPoolExecutor
# TODO: loop accept() และ pool.submit(handle, conn, addr)
```

ทดสอบด้วย telnet

เปิด terminal หลายหน้าต่าง แล้วรัน:

```
telnet 127.0.0.1 9002

# หน้าต่างแรก พิมพ์ nickname: Alice
# หน้าต่างที่สอง พิมพ์ nickname: Bob
# Alice พิมพ์: Hello Bob!
# Bob ควรเห็น: Alice: Hello Bob!
```

*** ให้รันไฟล์ Server ก่อน จากนั้นเปิด Terminal ไว้ 3 หน้าต่าง โดยใช้วิธีที่ Hint แนะนำในการเชื่อมต่อจาก Client ทั้ง 3 Terminal

Hint: ถ้า telnet ไม่มีในเครื่อง ให้ใช้ nc (netcat): nc 127.0.0.1 9002 หรือเขียน client Python อย่างง่ายแทน

ผลลัพธ์ที่คาดหวัง

```
PS C:\Users\point\OneDrive - Suranaree University of Technology\เอกสารการเรียนการสอน\1101180 - DGT01 1000 Network module\DGT01 1020 - network programming\Lab03\fileLab\Lab03> python task3_client.py

Enter nickname:Bob
Hi Alice!
Alice: Hello Bob!
Alice: How are you?
[Alice left]
█

PS C:\Users\point\OneDrive - Suranaree University of Technology\เอกสารการเรียนการสอน\1101180 - DGT01 1000 Network module\DGT01 1020 - network programming\Lab03\fileLab\Lab03> python task3_client.py

Enter nickname:Alice
[Bob joined]
Bob: Hi Alice!
Hello Bob!
How are you?
PS C:\Users\point\OneDrive - Suranaree University of Technology\เอกสารการเรียนการสอน\1101180 - DGT01 1000 Network module\DGT01 1020 - network programming\Lab03\fileLab\Lab03> █

PS C:\Users\point\OneDrive - Suranaree University of Technology\เอกสารการเรียนการสอน\1101180 - DGT01 1000 Network module\DGT01 1020 - network programming\Lab03\fileLab\Lab03> python task3_client.py

Enter nickname:Caesar
[Alice joined]
[Bob joined]
Bob: Hi Alice!
Alice: Hello Bob!
Alice: How are you?
[Alice left]
█
```

*** ตัวอย่าง ในรูปด้านบน จะทำการรัน Terminal ตัวที่ 3 ก่อน(นับเริ่มจากซ้าย นับหน้าต่าง Terminal ซ้ายสุด เป็นหน้าต่างที่ 1) จากนั้นรัน Terminal ที่ 2 และรัน Terminal ที่ 1 เป็นตัวสุดท้าย เมื่อรัน Terminal ที่ 1 เป็นตัวสุดท้ายแล้วนั้น ที่ Terminal 2 จะมีชื่อของคนแรกที่รันขึ้นมา จากนั้นให้เริ่มลองแชทสนทนา

เงื่อนไขผ่าน

- ✓ R1: client ส่ง nickname ก่อนเริ่มคุย
- ✓ R2: broadcast ในรูปแบบ nickname: ข้อความ
- ✓ R3: แจ้งทุกคนเมื่อมีคนเข้า/ออก
- ✓ R4: ใช้ threading.Lock() ทุกครั้งที่อ่าน/เขียน clients dict
- ✓ R5: ทดสอบด้วย telnet 3 หน้าต่างพร้อมกันได้สำเร็จ

Optional — ถ้าทำเสร็จก่อนเวลา

เพิ่ม command พิเศษใน chat server:

/list — แสดงรายชื่อทุกคนที่ online
/nick <ชื่อใหม่> — เปลี่ยน nickname
/quit — ออกจาก chat อย่างถูกต้อง



การส่งงาน

ส่งเป็นไฟล์ .py แยกตาม exercise: task1_echo.py, task2_pool.py, task3_chat.py

ทดสอบรันให้ผ่านทุก ‘เงื่อนไขการตรวจผ่าน’ ก่อนเรียก TA ตรวจ

ส่งผ่านระบบ Labpoint ภายในเวลาที่กำหนด **ก่อน 16.40 น.**